

**Министерство образования Российской Федерации**  
**Пензенский государственный университет Кафедра «Вычислительная**  
**техника»**

**Отчёт**

по лабораторной работе №10

по курсу «Л и О А в ИЗ»

на тему «Поиск расстояний во взвешенном графе»

Выполнили студенты группы  
24ВВВ4:

Кондратьев С.В.

Кошелев Р.Д.

Приняли к.т.н., доцент:

Юрова О.В.

к.э.н., доцент:

Акифьев И.В.

Пенза 2025

**Цель:** освоить практические навыки работы с динамическими структурами данных на языке С.

### Поиск расстояний во взвешенном графе

#### Общие сведения.

Во взвешенном графе в отличие от не взвешенного каждое ребро имеет вес, отличный от нуля. Поэтому в матрице смежности взвешенного графа содержится информация не только о наличии ребра, но и о его весе.

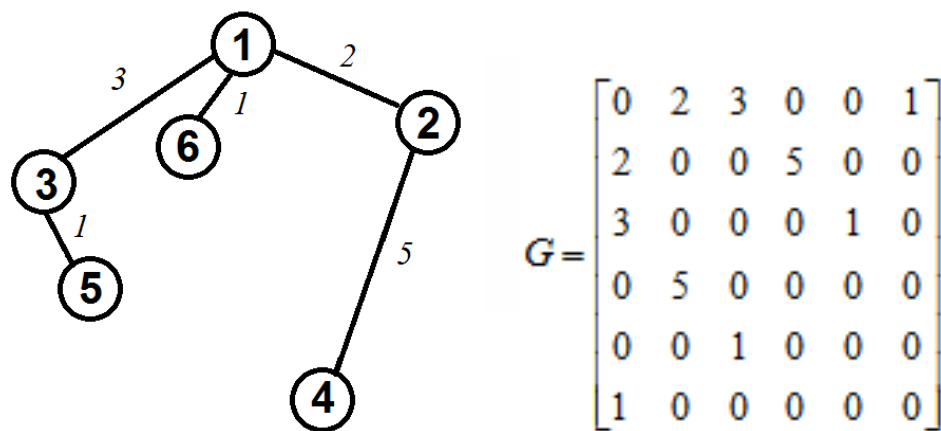


Рисунок 1 – Граф

Поиск расстояний между вершинами в таком графе также возможно построить используя процедуры обхода графа. Отличие от поиска расстояний в не взвешенном графе будет состоять в том, что при обновлении расстояния до вершины при ее посещении оно будет увеличиваться не на 1, а на величину веса ребра.

Таким образом, можно предложить следующую реализацию алгоритма обхода в ширину.

**Вход:**  $G$  – матрица смежности графа,  $v$  – исходная вершина.

**Выход:**  $DIST$  – вектор расстояний до всех вершин от исходной.

#### Алгоритм ПОШ

1.1. для всех  $i$  положим  $DIST[i] = -1$  пометим как "не посещенную";

1.2. **ВЫПОЛНЯТЬ**  $BFS(D, v)$ .

1.3 для всех  $i$  вывести  $DIST[i]$  на экран;

**АлгоритмBFSD( $v$ ):**

2.1. Создать пустую очередь  $Q = \{\}$ ;

2.2. Поместить  $v$  в очередь  $Q.push(v)$ ;

2.3. Обновить вектор расстояний  $DIST[x] = 0$ ;

2.4. **ПОКА**  $Q \neq \emptyset$  очередь не пуста **ВЫПОЛНЯТЬ**

2.5.  $v = Q.front()$  установить текущую вершину;

2.6. Удалить первый элемент из очереди  $Q.pop()$ ;

2.7. вывести на экран  $v$ ;

2.8. **ДЛЯ**  $i = 1$  **ДО**  $size\_G$  **ВЫПОЛНЯТЬ**

2.9. **ЕСЛИ**  $G(v,i) > 0$  **И**  $DIST[i] = -1$

2.10. **ТО**

2.11. Поместить  $i$  в очередь  $Q.push(i)$ ;

2.12. Обновить вектор расстояний  $DIST[i] = DIST[v] + G(v,i)$ ;

Реализация состоит из подготовительной части, в которой все вершины помечаются как не посещенные (п.1.1). Не посещенные вершины помечаются  $-1$ , т.к. значение  $0$  и  $1$  могут быть расстояниями. Расстояние  $0$  – от исходной вершины до самой себя.

В самой процедуре сначала создается пустая очередь (п. 2.1), в которую помещается исходная вершина, из которой начат обход (п.2.2). Расстояние до этой вершины (п.2.3) устанавливается равным  $0$  (расстояние до самой себя).

Далее итерационно, пока очередь не опустеет, из нее извлекается первый элемент, который становится текущей вершиной (п. 2.5, 2.6). Затем в цикле просматривается  $v$ -я строка матрицы смежности графа  $G(v,i)$ . Как только алгоритм встречает смежную с  $v$  не посещенную вершину (п.2.9), эта вершина помещается в очередь (п.2.11) и для нее обновляется вектор расстояния (п.2.12). Расстояние до новой  $i$ -й вершины вычисляется как

расстояние до текущей  $v$ -й вершины плюс вес ребра до новой вершины  $G(v,i)$ .

После просмотра строки матрицы смежности алгоритм делает следующую итерацию цикла 2.4 или заканчивает работу, если очередь пуста.

Если для всех пар вершин графа определены расстояния, то можно вычислить эксцентриситет

Если  $G$  - граф, содержащий непустое множество  $n$  вершин  $V$  и множество ребер  $E$  и  $d(v_i, v_j)$  – расстояние между двумя произвольными вершинами  $v_i$  и  $v_j$ , тогда для фиксированной вершины  $v$  величина

$$e(v) = \max d(v, v_j),$$

где  $v, v_j \in V$  и  $j = 1 \dots n$  называется **эксцентриситетом** вершины  $v_i$ .

Другими словами **эксцентриситет** вершины – расстояние до наиболее удаленной вершины графа.

Максимальный эксцентриситет среди эксцентриситетов всех вершин графа называется **диаметром** графа  $G$  и обозначается через  $D(G)$ .

Вершина  $v_i$  называется **периферийной**, если её эксцентриситет равен диаметру графа  $e(v_i) = D(G)$ .

Минимальный из эксцентриситетов вершин графа называется его **радиусом** и обозначается через  $r(G)$ .

Вершина  $v_i$  называется **центральной**, если её эксцентриситет равен радиусу графа  $e(v_i) = r(G)$ .

Множество всех центральных вершин графа называется его **центром**. Граф  $G$  может иметь единственную центральную вершину или несколько центральных вершин.

## Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного взвешенного графа  $G$ . Выведите матрицу на экран.

2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.

3.\* Сгенерируйте (используя генератор случайных чисел) матрицу смежности для ориентированного взвешенного графа  $G$ . Выведите матрицу на экран и осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием.

## Задание 2

1. Для каждого из вариантов сгенерированных графов (ориентированного и не ориентированного) определите радиус и диаметр.
2. Определите подмножества периферийных и центральных вершин.

## Задание 3\*

1. Модернизируйте программу так, чтобы получить возможность запуска программы с параметрами командной строки (см. описание ниже). В качестве параметра должны указываться тип графа (взвешенный или нет) и наличие ориентации его ребер (есть ориентация или нет).

Примечание: задания, помеченные символом \* выполняются по указанию преподавателя.

## Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
#include <string.h>
#include <ctype.h>
#include <limits.h>

#define MAXN 100
#define INF INT_MAX/4

// ----- Очередь (для простых случаев, но для взвешенных используем
Dijkstra) -----
typedef struct {
    int data[MAXN];
    int head, tail;
} Queue;

void initQueue(Queue* q) {
    q->head = q->tail = 0;
}
```

```

int isEmpty(Queue* q) {
    return q->head == q->tail;
}

void push(Queue* q, int x) {
    q->data[q->tail++] = x;
}

int pop(Queue* q) {
    return q->data[q->head++];
}

// ----- Dijkstra для взвешенных (и невзвешенных) графов -----
---
void Dijkstra(int start, int n, int G[MAXN][MAXN], int DIST[MAXN]) {
    int used[MAXN];
    for (int i = 0; i < n; i++) {
        DIST[i] = INF;
        used[i] = 0;
    }
    DIST[start] = 0;

    for (int iter = 0; iter < n; iter++) {
        int v = -1;
        int best = INF;
        for (int i = 0; i < n; i++) {
            if (!used[i] && DIST[i] < best) {
                best = DIST[i];
                v = i;
            }
        }
        if (v == -1) break;
        used[v] = 1;
        for (int to = 0; to < n; to++) {
            if (G[v][to] > 0) { // ребро существует (веса > 0)
                if (DIST[to] > DIST[v] + G[v][to]) {
                    DIST[to] = DIST[v] + G[v][to];
                }
            }
        }
    }

    // Приводим INF к -1 для совместимости с остальным кодом (если нужно)
    for (int i = 0; i < n; i++) {
        if (DIST[i] == INF) DIST[i] = -1;
    }
}

// Обёртка: если хотите, можно вызывать Dijkstra и получать -1 для недостижимых
void SHORTEST_DIST(int start, int n, int G[MAXN][MAXN], int DIST[MAXN]) {
    // внутренний массив с INF, потом переводим в -1
    int temp[MAXN];
    for (int i = 0; i < n; i++) temp[i] = INF;
    // Выполним Dijkstra используя temp, затем перепишем: INF -> -1
    int used[MAXN];
    for (int i = 0; i < n; i++) {
        temp[i] = INF;
        used[i] = 0;
    }
    temp[start] = 0;
    for (int iter = 0; iter < n; iter++) {
        int v = -1;
        int best = INF;
        for (int i = 0; i < n; i++) {
            if (!used[i] && temp[i] < best) {
                best = temp[i];
                v = i;
            }
        }
    }
}

```

```

    }
    }
    if (v == -1) break;
    used[v] = 1;
    for (int to = 0; to < n; to++) {
        if (G[v][to] > 0) {
            if (temp[to] > temp[v] + G[v][to])
                temp[to] = temp[v] + G[v][to];
        }
    }
}
for (int i = 0; i < n; i++) {
    if (temp[i] == INF) DIST[i] = -1;
    else DIST[i] = temp[i];
}
}

// ----- Генерация графов -----
void generateUndirected(int n, int G[MAXN][MAXN]) {
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int w;
            if (i == j)
                w = rand() % 2; // петля случайно 0 или 1
            else
                w = rand() % 10; // вес 0-9
            G[i][j] = w;
            G[j][i] = w;
        }
    }
}

void generateDirected(int n, int G[MAXN][MAXN]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                G[i][j] = rand() % 2; // петля случайно
            else
                G[i][j] = rand() % 10;
        }
    }
}

void generateUndirectedUnweighted(int n, int G[MAXN][MAXN]) {
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int w = rand() % 2; // 0 или 1
            G[i][j] = w;
            G[j][i] = w;
        }
    }
}

void generateDirectedUnweighted(int n, int G[MAXN][MAXN]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            G[i][j] = rand() % 2; // 0 или 1
        }
    }
}

// ----- Вывод -----
void printMatrix(int n, int G[MAXN][MAXN]) {
    for (int i = 0; i < n; i++) {
        for (int j =
0; j < n; j++)

```

```

        printf("%2d ", G[i][j]);
        printf("\n");
    }
}

// ----- Безопасный ввод -----
int safeInputInt(int maxVal) {
    char buffer[100];
    int value;
    int valid;

    while (1) {
        valid = 1;
        if (!fgets(buffer, sizeof(buffer), stdin)) return 0;
        buffer[strcspn(buffer, "\n")] = 0;

        if (strlen(buffer) == 0) valid = 0;
        else {
            for (int i = 0; i < (int)strlen(buffer); i++) {
                if (!isdigit((unsigned char)buffer[i])) valid = 0;
            }
        }

        if (valid) {
            value = atoi(buffer);
            if (value > 0 && value <= maxVal) return value;
        }
        printf("Неверный ввод! Введите число от 1 до %d: ", maxVal);
    }
}

int safeInputVertex(int n) {
    char buffer[100];
    int value;
    int valid;

    while (1) {
        valid = 1;
        if (!fgets(buffer, sizeof(buffer), stdin)) return 0;
        buffer[strcspn(buffer, "\n")] = 0;

        if (strlen(buffer) == 0) valid = 0;
        else {
            for (int i = 0; i < (int)strlen(buffer); i++) {
                if (!isdigit((unsigned char)buffer[i])) valid = 0;
            }
        }

        if (valid) {
            value = atoi(buffer);
            if (value >= 0 && value < n) return value; // диапазон 0..n-1
        }
        printf("Неверный ввод! Введите число от 0 до %d: ", n - 1);
    }
}

int safeInputZeroOne() {
    char buffer[100];
    int value;
    int valid;

    while (1) {
        valid = 1;
        if (!fgets(buffer, sizeof(buffer), stdin)) return 0;
        buffer[strcspn(buffer, "\n")] = 0;

        if (strlen(buffer) == 0) valid = 0;
        else {
            for (int i = 0; i < (int)strlen(buffer); i++) {

```



```

        if (!isdigit((unsigned char)buffer[i])) valid = 0;
    }
}

if (valid) {
    value = atoi(buffer);
    if (value == 0 || value == 1) return value;
}
printf("Неверный ввод! Введите 0 или 1: ");
}
}

// ----- Эксцентриситет, радиус, диаметр -----
void calcEccentricity(int n, int G[MAXN][MAXN], int ecc[MAXN]) {
    int DIST[MAXN];
    for (int i = 0; i < n; i++) {
        SHORTEST_DIST(i, n, G, DIST);
        int maxDist = -1;
        for (int j = 0; j < n; j++) {
            if (DIST[j] > maxDist)
                maxDist = DIST[j];
        }
        ecc[i] = maxDist;
    }
}

void calcGraphProperties(int n, int G[MAXN][MAXN]) {
    int ecc[MAXN];
    calcEccentricity(n, G, ecc);

    // Вывод эксцентрисета каждой вершины
    printf("\nЭксцентриситеты вершин:\n");
    for (int i = 0; i < n; i++)
        printf("Вершина %d: %d\n", i, ecc[i]);

    // Определяем радиус и диаметр (учитываем -1 как недостижимость)
    int radius = -1, diameter = -1;
    // Найдём первый ненулевой (не -1) элемент для инициализации
    for (int i = 0; i < n; i++) {
        if (ecc[i] >= 0) { radius = ecc[i]; diameter = ecc[i]; break; }
    }
    if (radius == -1) {
        printf("\nГраф неисполним для определения радиуса/диаметра (возможно, не свя-
зен).\n");
        return;
    }
    for (int i = 0; i < n; i++) {
        if (ecc[i] >= 0) {
            if (ecc[i] < radius) radius = ecc[i];
            if (ecc[i] > diameter) diameter = ecc[i];
        }
    }

    printf("\nРадиус графа: %d\n", radius);
    printf("Диаметр графа: %d\n", diameter);

    // Центральные вершины
    printf("Центральные вершины: ");
    for (int i = 0; i < n; i++)
        if (ecc[i] == radius)
            printf("%d ", i);
    printf("\n");

    // Периферийные вершины
    printf("Периферийные вершины: ");
    for (int i = 0; i < n; i++)
        if (ecc[i] == diameter)
            printf("%d ", i);

```

```

    printf("\n");
}

// ----- Наглядный вывод расстояний и эксцентриситета -----
void printDistancesAndEccentricity(int n, int G[MAXN][MAXN]) {
    int DIST[MAXN];

    for (int v = 0; v < n; v++) {
        SHORTEST_DIST(v, n, G, DIST);
        int ecc = 0;
        printf("\nВершина %d:\n", v);
        for (int u = 0; u < n; u++) {
            printf("    Расстояние до вершины %d: %d\n", u, DIST[u]);
            if (DIST[u] > ecc) ecc = DIST[u];
        }
        printf("    Эксцентриситет вершины %d: %d\n", v, ecc);
    }
}

// ----- Центр тяжести графа -----
void calcCenterOfMass(int n, int G[MAXN][MAXN]) {
    int DIST[MAXN];
    long long sumDist[MAXN];

    for (int i = 0; i < n; i++) sumDist[i] = (long long)INF;

    for (int v = 0; v < n; v++) {
        SHORTEST_DIST(v, n, G, DIST);
        long long s = 0;
        int unreachable = 0;
        for (int u = 0; u < n; u++) {
            if (DIST[u] == -1) {
                unreachable = 1;
                break;
            } else {
                s += DIST[u];
            }
        }
        if (!unreachable) sumDist[v] = s;
        else sumDist[v] = (long long)INF; // недостижима какая-то вершина
    }

    // Найдём минимум среди достижимых вершин
    long long best = (long long)INF;
    for (int i = 0; i < n; i++) {
        if (sumDist[i] < best) best = sumDist[i];
    }

    if (best == (long long)INF) {
        printf("\nНет вершины, из которой достижимы все остальные вершины. Невозможно
определить центр тяжести.\n");
        return;
    }

    printf("\nСуммы расстояний от вершин до всех остальных:\n");
    for (int i = 0; i < n; i++) {
        if (sumDist[i] == (long long)INF)
            printf("Вершина %d: недостижимы некоторые вершины\n", i);
        else
            printf("Вершина %d: %lld\n", i, sumDist[i]);
    }

    printf("\nЦентр(ы) тяжести (минимальная сумма = %lld): ", best);
    for (int i = 0; i < n; i++) {
        if (sumDist[i] == best) printf("%d ", i);
    }
    printf("\n");
}

```

```

// ----- Основная программа -----
int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "");
    srand((unsigned int)time(NULL));

    int n;
    int G_und[MAXN][MAXN];
    int G_dir[MAXN][MAXN];
    int DIST[MAXN];
    int und_ready = 0, dir_ready = 0;
    int weighted = 0;    // 1 – взвешенный, 0 – невзвешенный
    int directed = 0;    // 1 – ориентированный, 0 – неориентированный

    // Инициализация матриц нулями
    for (int i = 0; i < MAXN; i++)
        for (int j = 0; j < MAXN; j++) {
            G_und[i][j] = 0;
            G_dir[i][j] = 0;
        }

    // ----- Обработка параметров командной строки -----
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-weighted") == 0 && i + 1 < argc) {
            weighted = atoi(argv[i + 1]);
            i++;
        }
        else if (strcmp(argv[i], "-directed") == 0 && i + 1 < argc) {
            directed = atoi(argv[i + 1]);
            i++;
        }
    }

    printf("Введите количество вершин графа (1-100): ");
    n = safeInputInt(MAXN);

    // ----- Автоматическая генерация графа через параметры -----
    if (argc > 1) {
        if (directed) {
            if (weighted)
                generateDirected(n, G_dir);
            else
                generateDirectedUnweighted(n, G_dir);
            dir_ready = 1;
            printf("Ориентированный граф создан через параметры командной строки!\n");
        }
        else {
            if (weighted)
                generateUndirected(n, G_und);
            else
                generateUndirectedUnweighted(n, G_und);
            und_ready = 1;
            printf("Неориентированный граф создан через параметры командной строки!\n");
        }
    }

    // ----- Основное меню -----
    int choice, start;
    while (1)
    {
        printf("\n===== МЕНЮ =====\n");
        printf("1. Сгенерировать НЕориентированный граф\n");
        printf("2. Сгенерировать ориентированный граф\n");
        printf("3. Вывести НЕориентированный граф\n");
        printf("4. Вывести ориентированный граф\n");
        printf("5. Найти расстояния (неориентированный)\n");
        printf("6. Найти расстояния (ориентированный)\n");
    }
}

```

```

printf("7. Выход\n");
printf("8. Свойства неориентированного графа (радиус, диаметр, центры, перифе-
рия)\n");
printf("9. Свойства ориентированного графа (радиус, диаметр, центры, перифе-
рия)\n");
printf("10. Показать эксцентрисеты вершин\n");
printf("11. Центр тяжести графа (вершина(ы) с минимальной суммой расстояний)\n");
printf("Ваш выбор: ");
choice = safeInputInt(11);

if (choice == 1) {
    printf("Сгенерировать взвешенный (1) или невзвешенный (0) граф? ");
    int w = safeInputZeroOne();
    if (w) generateUndirected(n, G_und);
    else generateUndirectedUnweighted(n, G_und);
    und_ready = 1;
    printf("Неориентированный граф создан!\n");
}
else if (choice == 2) {
    printf("Сгенерировать взвешенный (1) или невзвешенный (0) граф? ");
    int w = safeInputZeroOne(); // исправлено: использую safeInputZeroOne
    if (w) generateDirected(n, G_dir);
    else generateDirectedUnweighted(n, G_dir);
    dir_ready = 1;
    printf("Ориентированный граф создан!\n");
}
else if (choice == 3) {
    if (!und_ready) printf("Граф ещё не создан!\n");
    else printMatrix(n, G_und);
}
else if (choice == 4) {
    if (!dir_ready) printf("Граф ещё не создан!\n");
    else printMatrix(n, G_dir);
}
else if (choice == 5) {
    if (!und_ready) printf("Сначала создайте неориентированный граф!\n");
    else {
        printf("Введите стартовую вершину (0..%d): ", n - 1);
        start = safeInputVertex(n);
        SHORTEST_DIST(start, n, G_und, DIST);

        printf("\nРезультат DIST (неориентированный):\n");
        for (int i = 0; i < n; i++)
            printf("DIST[%d] = %d\n", i, DIST[i]);
    }
}
else if (choice == 6) {
    if (!dir_ready) printf("Сначала создайте ориентированный граф!\n");
    else {
        printf("Введите стартовую вершину (0..%d): ", n - 1);
        start = safeInputVertex(n);
        SHORTEST_DIST(start, n, G_dir, DIST);

        printf("\nРезультат DIST (ориентированный):\n");
        for (int i = 0; i < n; i++)
            printf("DIST[%d] = %d\n", i, DIST[i]);
    }
}
else if (choice == 8) {
    if (!und_ready) printf("Сначала создайте неориентированный граф!\n");
    else calcGraphProperties(n, G_und);
}
else if (choice == 9) {
    if (!dir_ready) printf("Сначала создайте ориентированный граф!\n");
    else calcGraphProperties(n, G_dir);
}
else if (choice == 10) {
    if (!und_ready) printf("Сначала создайте неориентированный граф!\n");

```

```

        else printDistancesAndEccentricity(n, G_und);
    }
    else if (choice == 11) {
        // Центр тяжести: можно для неориентированного и ориентированного графа от-
        дельно спрашивать,
        // но здесь реализуем оба варианта – спрашиваем пользователя
        printf("Для какого графа найти центр тяжести? 0 - неориентированный, 1 - ори-
        ентированный: ");
        int gtype = safeInputZeroOne();
        if (gtype == 0) {
            if (!und_ready) printf("Сначала создайте неориентированный граф!\n");
            else calcCenterOfMass(n, G_und);
        } else {
            if (!dir_ready) printf("Сначала создайте ориентированный граф!\n");
            else calcCenterOfMass(n, G_dir);
        }
    }
    else if (choice == 7) {
        printf("Выход.\n");
        break;
    }
    else {
        printf("Неверный выбор!\n");
    }
}

return 0;
}

```

## Результат работы программы

```
Введите количество вершин графа (1-100): 3

===== МЕНЮ =====
1. Сгенерировать НЕориентированный граф
2. Сгенерировать ориентированный граф
3. Вывести НЕориентированный граф
4. Вывести ориентированный граф
5. Найти расстояния (неориентированный)
6. Найти расстояния (ориентированный)
7. Выход
8. Свойства неориентированного графа (радиус, диаметр, центры, периферия)
9. Свойства ориентированного графа (радиус, диаметр, центры, периферия)
10. Показать наглядно DIST и эксцентрисеты вершин
Ваш выбор: |
```

Рисунок 1 – Начальное меню

```
Введите количество вершин графа (1-100): 3

===== МЕНЮ =====
1. Сгенерировать НЕориентированный граф
2. Сгенерировать ориентированный граф
3. Вывести НЕориентированный граф
4. Вывести ориентированный граф
5. Найти расстояния (неориентированный)
6. Найти расстояния (ориентированный)
7. Выход
8. Свойства неориентированного графа (радиус, диаметр, центры, периферия)
9. Свойства ориентированного графа (радиус, диаметр, центры, периферия)
10. Показать наглядно DIST и эксцентрисеты вершин
Ваш выбор: 1
Сгенерировать взвешенный (1) или невзвешенный (0) граф? |
```

Рисунок 2 – Выбор взвешенности графа

```
Неориентированный граф создан!

===== МЕНЮ =====
1. Сгенерировать НЕориентированный граф
2. Сгенерировать ориентированный граф
3. Вывести НЕориентированный граф
4. Вывести ориентированный граф
5. Найти расстояния (неориентированный)
6. Найти расстояния (ориентированный)
7. Выход
8. Свойства неориентированного графа (радиус, диаметр, центры, периферия)
9. Свойства ориентированного графа (радиус, диаметр, центры, периферия)
10. Показать наглядно DIST и эксцентрисеты вершин
Ваш выбор: 3
0 9 1
9 0 9
1 9 1
```

Рисунок 3 – Вывод графа

```
7. Выход
8. Свойства неориентированного графа (радиус, диаметр, центры, периферия)
9. Свойства ориентированного графа (радиус, диаметр, центры, периферия)
10. Показать наглядно DIST и эксцентрисеты вершин
Ваш выбор: 8

Эксцентриситеты вершин:
Вершина 0: 9
Вершина 1: 9
Вершина 2: 9

Радиус графа: 9
Диаметр графа: 9
Центральные вершины: 0 1 2
Периферийные вершины: 0 1 2
```

Рисунок 4 –Свойства графа

**Вывод:** в ходе выполнения данной лабораторной работы освоили практические навыки работы с динамическими структурами данных на языке С.